

Oracle Constructions: One Collapsing P and NP, One Separating Them

We prove two existence statements:

1. There exists an oracle A such that $P^A = NP^A$.
2. There exists an oracle B such that $P^B \neq NP^B$.

The first construction uses a PSPACE-complete oracle. The second uses a direct diagonalization against all deterministic polynomial-time oracle machines.

Part I. An oracle A with $P^A = NP^A$

Take A to be the language TQBF of true fully quantified Boolean formulas. Since TQBF is PSPACE-complete, standard complexity theory gives

$$P^{TQBF} = PSPACE.$$

So it is enough to show that NP^{TQBF} is also exactly PSPACE.

Why $P^{TQBF} = PSPACE$

If L is in PSPACE, then because TQBF is PSPACE-complete there is a polynomial-time many-one reduction f such that $x \in L$ iff $f(x) \in TQBF$. A polynomial-time oracle machine with oracle TQBF can compute $f(x)$, ask one oracle question, and decide whether x is in L . Hence $PSPACE \subseteq P^{TQBF}$.

The reverse inclusion $P^{TQBF} \subseteq PSPACE$ also holds because a polynomial-time oracle machine can make only polynomially many oracle queries, each of polynomial length, and each TQBF query can be answered in polynomial space. Simulating the entire oracle computation therefore uses only polynomial space. Thus $P^{TQBF} = PSPACE$.

Why $NP^{TQBF} = PSPACE$

First, $P^{TQBF} \subseteq NP^{TQBF}$ is immediate, since deterministic polynomial-time computation is a special case of nondeterministic polynomial-time computation.

For the other direction, let L be in NP^{TQBF} . Then there is a nondeterministic polynomial-time oracle machine M with oracle TQBF deciding L . We show L is in PSPACE.

On input x , the machine M explores a computation tree of depth at most $p(|x|)$ for some polynomial p . Along any branch, M writes only polynomially many bits and asks only polynomially many oracle questions, each of length polynomial in $|x|$. Every oracle question is a TQBF instance and can be answered in polynomial space. A depth-first search of the nondeterministic computation tree, evaluating each oracle query when it appears, uses only polynomial space overall.

Therefore $NP^{TQBF} \subseteq PSPACE$. Combining this with $P^{TQBF} = PSPACE$ gives

$$NP^{TQBF} \subseteq PSPACE = P^{TQBF} \subseteq NP^{TQBF}.$$

Hence $NP^{TQBF} = P^{TQBF}$.

So with $A = TQBF$ we obtain $P^A = NP^A$.

Part II. An oracle B with $P^B \neq NP^B$

We now build an oracle B and a language L_B such that L_B is in NP^B but not in P^B .

Define

$$L_B = \{ 1^n : \text{there exists a string } y \text{ of length } n \text{ with } y \in B \}.$$

In words: 1^n belongs to L_B exactly when B contains at least one string of length n.

Why L_B is in NP^B

An NP oracle machine, on input 1^n , nondeterministically guesses a string y of length n and asks the oracle whether y is in B. If some such y lies in B, one branch accepts; otherwise all branches reject. This takes polynomial time, so L_B is in NP^B for every oracle B.

Diagonalizing against every polynomial-time oracle machine

Enumerate all deterministic polynomial-time oracle Turing machines as $M_1, M_2, M_3, \dots$. We may assume that machine M_i runs in time at most $n^i + i$ on inputs of length n. We construct B in stages so that M_i fails to decide L_B on at least one input.

At stage i, choose a fresh length $n = n_i$ so large that

$$n^i + i < 2^n.$$

Also choose n_i larger than all previously used lengths, so later stages do not affect the truth of earlier diagonalization decisions.

Now run M_i^B on input 1^n , answering all oracle queries of length n with NO for the moment, and keeping all answers for other lengths consistent with what has already been fixed in previous stages. Since M_i runs in time less than 2^n , it can query fewer than 2^n strings of length n. But there are exactly 2^n such strings, so at least one length-n string is never queried.

After the simulation finishes, there are two cases.

Case 1: $M_i^B(1^n)$ rejects. Then place one unqueried string z of length n into B. Now, by definition, 1^n belongs to L_B because B contains a length-n string, but M_i rejected. So M_i is wrong on 1^n .

Case 2: $M_i^B(1^n)$ accepts. Then place no string of length n into B. Now 1^n does not belong to L_B , but M_i accepted. Again M_i is wrong on 1^n .

In either case, stage i ensures that M_i does not decide L_B correctly.

Why later stages do not spoil earlier ones

At stage i we decide membership in B only for strings of one new length n_i . Since L_B on input 1^{n_i} depends only on whether B contains a string of length n_i , future changes at different lengths cannot change the truth value of $1^{n_i} \in L_B$. Thus each diagonalization requirement, once met, stays met forever.

Conclusion

The final oracle B satisfies $L_B \in \text{NP}^B$, but no deterministic polynomial-time oracle machine decides L_B . Therefore L_B is not in P^B . Hence

$$P^B \neq \text{NP}^B.$$

Final statement

We have shown both existence claims:

- There exists an oracle A (for example, $A = \text{TQBF}$) such that $P^A = \text{NP}^A$.
- There exists an oracle B such that $P^B \neq \text{NP}^B$.

This is the classical relativization phenomenon: oracle access can make the P versus NP relationship collapse in one world and separate in another.